

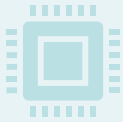
Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Introdução a Linguagem C

Prof. Dr. Rogério Pinto Alexandre
rogerio.alexandre@fatec.sp.gov.br
@rogerio.alexandre_

Fatec
Lins
Prof. Antonio Seabra

Breve Histórico



Em 1973 Dennis Ritchie reescreveu o sistema operacional UNIX em uma linguagem de alto nível (Linguagem C).



Surgem diversas implementações utilizando C, porém os códigos gerados eram incompatíveis.



ANSI (American National Standard Institute), em 1983 padronizou a linguagem C.

Características da Linguagem C



Portabilidade entre máquinas e sistemas operacionais.



Alia características de linguagens de alto e baixo nível (liberdade ao programador).



Programas Estruturados.

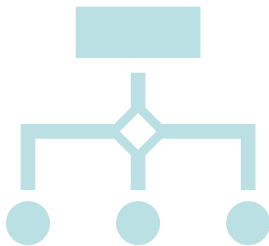


Total interação com o Sistema Operacional.

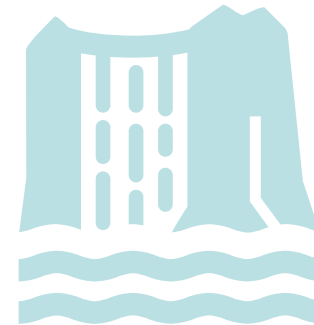


Código compacto e rápido, quando comparado ao código de outras linguagens de complexidade análoga.

Características da Linguagem C



C é uma linguagem compilada: lê todo o código fonte e gera o código objeto (linguagem de máquina) uma única vez.



Linguagens Interpretadas: lê o código fonte, traduz e executa cada vez que o programa for executado.

Características da Linguagem C



A linguagem C baseia-se no conceito de blocos construtivos.



Os blocos construtivos são denominados **funções**. Um programa C é, portanto, uma coletânea de uma ou mais funções.



Assim, para escrever um programa C, primeiro deve-se criar as funções para depois juntá-las.



Uma função é uma sub-rotina formada por uma ou mais declarações C, que executa uma ou mais subtarefas.

Detalhes Importantes



Sempre que o código fonte for alterado ele deve ser novamente compilado.



C é “case sensitive”



Deve-se listar antecipadamente todas as variáveis utilizadas no programa.



Os blocos são delimitados por abre-chave “{”, para indicar o início do bloco e fecha-chave “}” para indicar o fim do bloco;

Palavras reservadas

São palavras que têm um significado especial para a linguagem.



**auto, break, case, if, for, while,
continue, return, const,....**



C entende tais palavras apenas em letras minúsculas.

Bibliotecas



Conjunto de funções para realizar tarefas específicas.



Biblioteca padrão C - ANSI - funções básicas.



As primeiras linhas do programa indicam as bibliotecas utilizadas:

`#include "minha_biblioteca.h"` ou
`#include <minha_biblioteca.h>`



Exemplo:

`#include <stdio.h>`
`#include <stdlib.h>`

A função `main()`



Todo programa em C é um conjunto de funções, inclusive o programa principal que tem um identificador reservado denominado `main()`.



A função **`main()`** é especial por ser a primeira a ser chamada quando seu programa é executado. Significa o início de seu programa.



É obrigatória a existência de `main()` em algum ponto do programa, para que o compilador C consiga determinar onde iniciar a execução.

Estrutura de um programa C

```
/* definições de pré-processamento */  
.  
.  
.  
/* declarações de variáveis globais */  
.  
.  
.  
int main( )  
{  
/* declarações de variáveis locais */  
.  
.  
.  
/* corpo do programa principal */  
.  
.  
.  
}  
/* definição de outras funções */  
.  
.  
.
```

Modelo de um programa C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello, World!");
```

```
    return 0;
```

```
}
```

Variável

- **Variável:** posição nomeada de memória, usada para guardar um valor que pode ser modificado pelo programa;
- Espaço de memória de um certo tipo de dado associado a um nome para referenciar seu conteúdo.

```
int main (void)
{
    int idade;
    idade = 30;
    printf (" A idade é : %d", idade);
}
```

Identificadores

Os **identificadores** nomeiam as variáveis, as constantes, tipos das variáveis, funções e até mesmo o controle do fluxo de um programa escrito em C.

Quantos caracteres quiser (32);

Comece com letras ou sublinhado:

- Seguidos de letras, números ou sublinhados

C é sensível ao caso:

- peso <> Peso <> pEso

Não podemos definir um identificador com o mesmo nome que uma palavra chave

- auto static extern int long if while do

Declaração de Variáveis

- Instrução para reservar uma quantidade de memória para um certo tipo de dado, indicando o nome pelo qual a área será referenciada.
- Deve obedecer ao seguinte formato:
 <tipo> nome-da-variável; ou
 <tipo> nome1, nome2,...,nome_n;

Exemplo:

char nome;

int idade, num;

Tipos Básicos de Dados

- Para declarar uma variável em C pode-se usar os tipos primitivos pré-determinados pela linguagem. Para tal considere a tabela:

tipo	bytes	escala
char	1	-128 a 127
int	2	-32.768 a 32.767
float	4	3.4e-38 a 3.4e+38
double	8	1.7e-308 a 1.7e+308
void	0	sem valor

Tipos de dados

O tipo **void** é também chamado tipo vazio, não contém nenhum valor. Ou seja, o conjunto de valores assumidos pelo tipo void é vazio, logo não existem variáveis deste tipo. Ele é usado para passagem de parâmetros nulos para funções, para indicar que as funções não retornam valores e na criação de ponteiros genéricos.

Embora seja usado para representar caracteres, o tipo **char** também armazena valores. Isso porque em C um caracter (como uma letra, por exemplo), é representado por seu valor ASCII.

Na linguagem C, **não existe o tipo lógico** (como por exemplo, no PASCAL existe o tipo Boolean), **mas existem valores lógicos**. Foi convencionalizado que **0** é falso e qualquer valor não nulo é verdadeiro.

Exemplo

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    char ch; //Declara ch como tipo char
    ch='A'; //Atribui o caracter a para a variável ch
    ch=ch+1; // OK! pois o valor decimal de ch é 65
    printf("Caracter representado por ch=%c, cujo valor
    ASCII é= %d \n",ch,ch);
    system("pause");
}
```

Ao término do programa será exibida na tela a seguinte mensagem:

Caracter representado por ch=B, cujo valor ASCII é=66

Inicialização de variáveis

- Em C, existe a possibilidade de declararmos uma variável e inicializá-la com um certo valor, um valor inicial.

```
#include <stdio.h>
int main(void)
{
    int soma=10;
    float money=2.21;
    char letra= 'A';
    double pi=2.01E6;

    printf ("valor da soma = %d\n", soma);
    printf ("Valor de Money = %f\n", money);
    printf("Valor de Letra = %c\n", letra);
    printf("Valor de Pi = %e\n", pi);
}
```

Operador de Atribuição

- Atribui um valor a uma variável. É usado o símbolo **=**.

```
#include <stdio.h>
int main( ) {
int a, b = 10, c, d;
a = b * 2;
c = d = a;
printf("\n a=%d \n b=%d \n c=%d \n d=%d", a, b, c, d);
}
```

Operadores Aritméticos

Operador	Ação
+	Adição
*	Multiplicação
/	Divisão
%	Resto da divisão inteira
-	Subtração
--	Decremento
++	Incremento

Exemplo

```
#include <stdio.h>
int main( )
{
    int a,b,c,d;
    a = 10;
    b = c = a*2;
    d = c%a;
    printf("\n a=%d \n b=%d \n c=%d \n d=%d",a,b,c,d);
}
```

Operadores de Incremento e Decremento

Estes operadores quando aplicados a uma variável, incrementam (++) em 1 ou decrementam (--) em 1 o valor da variável.



Estes operadores trabalham em dois modos: o **pós-fixado**, quando o operador vem depois da variável e o **pré-fixado**, quando o operador vem antes da variável.



A diferença entre os dois modos é que:

no **pós-fixado** o valor da variável é usado e depois seu valor é incrementado (ou decrementado);

no **pré-fixado** o valor da variável é incrementado (ou decrementado) antes de seu valor ser usado.

Exemplo 1

```
#include <stdio.h>
int main( )
{
    int a=0,b=1;
    a++; //Equivalente a fazer: a=a + 1
    b--; //Equivalente a fazer: b=b - 1
    printf("\n Valor de a=%d \n Valor de b=%d", a,b);
}
```

Exemplo 2

```
#include <stdio.h>
int main( )
{
    int x, y=0;
    x= y++; //Após a execução as variáveis terão os valores: x=0 e y=1
    printf("\n Pos-incremento: valor de x=%d, valor de y=%d",x,y);
    y=0;
    x= ++y; //Após a execução as variáveis terão os valores: x=1 e y=1
    printf("\n Pre-incremento: valor de x=%d, valor de y=%d",x,y);
    y=1;
    x= y--; //Após a execução as variáveis terão os valores: x=1 e y=0
    printf("\n Pos-decremento: valor de x=%d, valor de y=%d",x,y);
    y=1;
    x= --y; //Após a execução as variáveis terão os valores: x=0 e y=0
    printf("\n Pre-decremento: valor de x=%d, valor de y=%d",x,y);
}
```

Operadores Relacionais e Lógicos

OPERADOR	AÇÃO
>	Maior que
>=	Maior ou igual que
<	Menor que
<=	Menor ou igual que
==	Igual a
!=	Diferente de
&&	Condição "E" (and)
	Condição "OU" (or)
!	Não (not)

Operadores Relacionais e Lógicos

Na linguagem C não existe tipo lógico, mas existem valores lógicos. Para as expressões que resultam em um valor lógico, ficou convencionalizado que quando o valor retornado pela expressão for **0(zero)** significa **FALSO** e quando for **1(um)** significa **VERDADEIRO**.

```
#include <stdio.h>
int main( ) {
int verdadeiro, falso;
verdadeiro = (15 < 20);
falso = (15 == 20);
printf("\n VERDADEIRO=%d, FALSO=%d", verdadeiro, falso);
}
```

Abreviações de Operações de Atribuição

- Existe a possibilidade de reduzirmos ou abreviarmos algumas operações onde o resultado da operação será atribuído a um de seus operandos. Por exemplo:

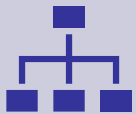
▪ $a += b$	o mesmo que $a = a + b$
▪ $a -= b$	o mesmo que $a = a - b$
▪ $a *= b$	o mesmo que $a = a * b$
▪ $a /= b$	o mesmo que $a = a / b$
▪ $a \% = b$	o mesmo que $a = a \% b$
▪ $a ++ = 3$	o mesmo que $a = a + 3$

Operador de Endereço &

```
#include <stdio.h>
int main ( )
{
    int num;
    num = 2;
    printf ("valor = %d,
endereço = %i",
num,&num);
}
```

- Quando usamos & precedendo uma variável estamos falando do endereço desta variável na memória.
- Saída: valor = 2, endereço = 1230
- Varia conforme memória da máquina

Funções de Entrada e Saída



Como a linguagem C é manipulada e estruturada sob a forma de funções, para que possamos efetuar uma entrada ou uma saída de dados será necessário o uso de funções.



No entanto são muitas as funções que efetuam tais operações, e as principais se encontram na biblioteca chamada **<stdio.h>**, a qual pode ser acessada através de **#include <stdio.h>**. Há outras bibliotecas, como **<stdlib.h>**, **<conio.h>**

Função printf()

- Tem a finalidade de enviar um conjunto de caracteres para a saída padrão (monitor).
- A sintaxe da função é:
printf("string de controle", lista de argumentos)

Exemplo:

```
# include <stdio.h>
int main ( ) {
    int idade;
    printf ("Digite a sua idade:\n");
    scanf ("%d", &idade);
    printf("Sua idade é: %d", idade);
}
```

Função printf()



Para imprimir expressões (ou variáveis) devemos efetuar a formatação de acordo com os valores desejados na impressão.



Entre o sinal de % e a máscara pode ser informado o número de caracteres a serem impressos.



Formatação da saída. Ex: **%5.2f**

Função printf() - Formatação

Máscara	Formatação da saída
%s	para string
%c	para caractere
%d	para decimal
%i	para int
%f	para float
%x	para hexadecimal
%e	para notação científica
%o	para octal
%u	para decimal sem sinal
%ld	para decimal longo
%lf	para ponto flutuante longo (double)

Função printf() - Exemplo

```
# include <stdio.h>
int main ( )
{
    int car = 'A';
    printf("\n O caractere %c \n ", car);
    printf(" E' em decimal %d \n ", car);
    printf(" e em hexadecimal %x \n ", car);
}
```

Função printf() – Caracteres especiais

- No exemplo anterior foi usado um caracter especial “\n” que significa <ENTER> após a impressão.
- Existem outros caracteres especiais como:

\a	beep
\b	retrocede uma posição
\\	o caractere \
\n	nova linha e outros
\t	coloca um TAB
\”	o caractere aspas (“)
\f	salta página de formulário

Função scanf()



Tem a finalidade de ler um conjunto de caracteres contidos no buffer do teclado. Para a leitura dos valores deve-se formatar cada valor de entrada.



A sintaxe é similar a função printf():



scanf(“expressão de controle”, lista de argumentos)

Função scanf() - Exemplo

```
#include <stdio.h>

int main( ) {
    char nome[25];
    int idade;
    printf("\n Entre com seu Nome:");
    fflush(stdin);
    scanf("%s", nome);
    printf("Entre com sua idade:");
    fflush(stdin);
    scanf("%d", &idade);
    printf("\n Olá %s, você tem %d anos!", nome, idade);
}
```

Função scanf() - Observações

- O símbolo **&** é utilizado para expressar o endereço de memória da variável que vai armazenar o valor entrado. Quando o valor entrado é uma string (vetor de caracteres) não é necessário colocar o **&**, pois a linguagem C já trata uma string pelo endereço de seu primeiro caractere;
- Como a leitura é feita no buffer do teclado é necessário limpá-lo antes de efetuar uma entrada, para isto utiliza-se a função:
fflush (stdin); // stdin é a entrada padrão
- A função **scanf()** não consegue fazer a leitura de uma string que contenha espaço em branco entre as palavras. Por exemplo, considere a execução do programa anterior como a seguir:
 - *Entre com seu nome: Maria Silva*
 - *Entre com sua idade: 22*
 - *Olá Maria, você tem 22 anos!*
- Verifique que o sobrenome (Silva) foi ignorado pela função scanf(). Portanto, ela não é recomendada para entrada de cadeia de caracteres.

Função puts()

- Tem a finalidade de enviar uma string para a saída padrão. Após imprimir a string, puts() já posiciona o cursor na próxima linha.

Exemplo:

```
# include <stdio.h>
int main ( ) {
char nome[10]="Antonio";
puts("Esta é uma string ! ");
puts(nome);
}
```

Função gets()

- Faz a leitura de uma string.

```
# include <stdio.h>
int main ( ) {
char nome [30];
puts ("\n Digite o seu nome: ");
fflush(stdin);
gets (nome);
printf ("\n Oi ... %s !", nome);
}
```

Função putchar()

- Tem a finalidade de imprimir um caractere na saída padrão na posição do cursor.
- Após a impressão o cursor permanece na posição seguinte ao caractere impresso.

```
# include <stdio.h>
int main ( ) {
char ch='A';
putchar(ch);
putchar(65); //Será exibido o character cujo código ASCII é 65
putchar('B');
}
```

Função getchar()

- Lê da entrada padrão um caractere.

```
# include <stdio.h>
int main ( )
{
    unsigned int car;
    printf ("\n Digite um caractere:");
    fflush(stdin);
    car = getchar();
    printf ("\n O caractere digitado = %c \n ", car);
}
```

Função getch()

- Lê um caractere da entrada padrão, ecoando-o na tela. A diferença desta função para a função getchar(), é que com getch() não é necessário digitar <ENTER> após digitar o caractere.

```
#include <stdio.h>
int main( ) {
    char ch;
    printf("\n Digite algum caractere: ");
    fflush(stdin);
    ch=getch( );
    printf("\n A tecla digitada foi %c", ch);
}
```

Função getch()

- Lê um caractere da entrada padrão, porém não ecoa o caractere na tela. Esta função é recomendada quando se deve esconder as teclas digitadas (no caso de senhas) ou tratar a tecla antes de ecoar na tela (no caso de editores de texto, por exemplo).
- **Observação:** Existem muitas outras funções de entrada e saída de dados pertencentes a diversas bibliotecas.

Função strcpy()

- **Sintaxe:**
 - `strcpy(string_destino, string_origem);`
- Realiza a cópia do conteúdo de uma variável a outra.
 - Obs: Ambas devem ser strings.
- É necessário usar a biblioteca `<string.h>`

Exemplo strcpy()

```
#include <stdio.h>
#include <string.h> //necessário para strcpy
#include <conio.h>
int main (void)
{
    char nome[15];
    strcpy(nome, "Fulano de Tal");
    //strcpy(string_destino, string_origem);
    //note que a string de destino é nome
    //a string de origem é "Fulano de Tal"
    printf("Nome = %s", nome);
    getch();
    return 0;
}
```

Função strcat()

- **Sintaxe:**
 - `strcat(string_destino, string_origem);`
- Realiza a concatenação do conteúdo de uma variável a outra.
 - Obs: Ambas devem ser strings.
- É necessário usar a biblioteca `<string.h>`

Função strlen()

- **Sintaxe:**
 - variável do tipo inteiro = `strlen(string)`;
- Determina o tamanho de uma string.
- É necessário usar a biblioteca `<string.h>`

Exemplo strlen()

```
#include <stdio.h>
#include <string.h> //necessário para strlen
#include <conio.h>
int main (void)
{
    char str[5] = "Curso";
    int tamanho;
    tamanho = strlen(str);
    printf("O tamanho da string %s vale %d\n", str, tamanho);
    getch();
    return 0;
}
```